



ENG-39 — Decisión: generator de Prisma (prisma-client-js)

Registro de Decisión Técnica (ADR)

VERSIÓN	1.0
FECHA	08/07/2026
SPRINT	1
ÉPICA	EP-10 (Plataforma e Infraestructura)
TIPO	Decisión técnica (ADR)
RELACIONADO	ENG-37 (spike RLS)

Tabla de Contenidos

1. Qué cambió
2. Contexto: los dos generators
3. Por qué se cambió a prisma-client-js
4. Impacto en el equipo
5. Cómo regenerar el cliente
6. Decisión
7. Checklist para el PR

Este documento acompaña al spike ENG-39 (setup de PostgreSQL en Docker). Registra el porqué del cambio de generator en `prisma/schema.prisma`, para que el equipo pueda confirmarlo y coordinarlo (lo pidió el review antes de mergear).

1. Qué cambió

En `prisma/schema.prisma`, el `provider` del generator del cliente:

```
generator client {  
-   provider = "prisma-client"  
+   provider = "prisma-client-js"  
    output   = "../generated/prisma"  
}
```

Nada más del schema cambió. El `output` sigue siendo `../generated/prisma` y el import en el código sigue igual:

```
import { PrismaClient } from '../generated/prisma';
```

2. Contexto: los dos generators

Prisma 6 ofrece dos generators para el cliente:

Generator	Qué es	Formato de salida
<code>prisma-client</code>	El generator nuevo (Prisma 6+), pensado para ESM y edge/serverless. Requiere <code>output</code> explícito.	ESM por defecto
<code>prisma-client-js</code>	El generator clásico y estable , el que usó Prisma durante años.	CommonJS-compatible

Ambos generan a un directorio (`generated/prisma`) y se importan igual. La diferencia relevante para nosotros es el **formato de módulo** de la salida.

3. Por qué se cambió a `prisma-client-js`

La suite de tests corre con **Jest + ts-jest**, que transpila y ejecuta en **CommonJS** (ver `test/jest-integration.json` → `transform: ts-jest`, y el `jest` config del `package.json`).

El generator nuevo (`prisma-client`) emite un cliente **ESM-first**, que al ser consumido desde el runtime CommonJS de ts-jest genera fricción de interoperabilidad (errores de import/ `exports`, `ERR_REQUIRE_ESM` y similares). El generator clásico (`prisma-client-js`) emite salida compatible con CommonJS, que ts-jest consume sin configuración extra.

Resultado empírico: con `prisma-client-js`, `prisma db push` sincroniza el schema y la suite de integración corre en verde (`1 passed`) sin tocar la config de Jest ni agregar `ts-node/esm`, `transform-ignore-patterns`, ni flags experimentales de ESM.

NOTA

Alternativa descartada (por ahora): quedarse en `prisma-client` (ESM) y adaptar ts-jest a ESM. Es viable pero mete complejidad (config ESM en Jest, posibles ajustes en `tsconfig` y en el resto del proyecto Nest, que hoy es CommonJS). Para un spike de infraestructura no se justifica; se puede reevaluar si el proyecto migra a ESM de forma integral.

4. Impacto en el equipo

- `generated/prisma` está en `.gitignore` (no se commitea). Cada desarrollador y CI regeneran el cliente localmente. Por eso el cambio de generator **no** se ve en un archivo generado versionado: se propaga solo cuando cada uno regenera.
- **Todos deben regenerar el cliente** después de traer esta rama (ver sección 5). Si alguien tiene un `generated/prisma` viejo (ESM) cacheado, puede ver errores de import hasta regenerar.
- **ENG-37 (RLS):** corre en paralelo y toca la misma capa de datos. En ENG-37 el cambio "generó y corrió sin problemas", pero **hay que coordinar el merge** para no pisarse en `schema.prisma` y para que ambos spikes queden sobre el mismo generator.

IMPORTANTE

Acción requerida: avisar a quien lleva ENG-37 antes de mergear, para coordinar el cambio en `schema.prisma` y que ambos spikes queden sobre el mismo generator.

5. Cómo regenerar el cliente

Después de checkout/pull de esta rama:

```
# regenera generated/prisma con el generator nuevo
pnpm exec prisma generate

# o, si además querés sincronizar el schema contra tu BD de test:
pnpm run test:integration # corre prisma db push + jest
```

En el devcontainer esto pasa solo: el `postStartCommand` corre `prisma db push`, que regenera el cliente.

6. Decisión

Adoptar `prisma-client-js` como generator del cliente mientras el proyecto (Nest + Jest/ts-jest) sea CommonJS. Es el camino estable, sin config extra y con la suite en verde. Revisar la decisión solo si se decide migrar el proyecto entero a ESM.

7. Checklist para el PR

- ☐ Documentado el porqué del cambio de generator (este archivo).
- ☐ Coordinado con quien lleva **ENG-37 (RLS)**.
- ☐ Avisado al equipo que hay que correr `pnpm exec prisma generate` tras traer la rama.